Optimizing Neural Speech Decoding on Consumer Hardware: A 10× Speedup through Architecture and Training Improvements

Jimmy Fang
ECE C143A
University of California, Los Angeles
Los Angeles, CA 90095
jimmyfang@ucla.edu
UID 906062643

Drew Wan
ECE C143A
University of California, Los Angeles
Los Angeles, CA 90095
drewkeithwan@ucla.edu
UID 305751876

Abstract

We present a comprehensive optimization study of the baseline GRU model for neural speech decoding on the Brain-to-Text Benchmark '24 dataset. Through systematic improvements spanning mixed-precision training, data loading efficiency, model architecture modifications, and learning rate scheduling, we achieve a 10.2× training speedup to reach sub-0.21 phoneme error rate (PER) while simultaneously improving final model accuracy from 0.2055 to 0.1718 PER (16.4% relative error reduction). Notably, all experiments were conducted on consumer-grade hardware, demonstrating that significant improvements in neural speech decoding are achievable without high-performance computing resources. Our architectural changes reduce model parameters from 56.7M to 35.2M (38% reduction) while improving both training efficiency and final accuracy, suggesting the original model was overparameterized for this task.

1 Introduction

Brain-computer interfaces (BCIs) that decode speech intention from neural signals offer transformative potential for individuals with speech and motor impairments caused by conditions such as amyotrophic lateral sclerosis (ALS), brainstem stroke, or traumatic brain injury (9). The Brain-to-Text Benchmark '24 (10) provides a standardized dataset and baseline model for advancing this field, featuring microelectrode array recordings from the ventral premotor cortex of a participant with ALS attempting to speak sentences. This cortical region has been shown to contain rich information about speech articulatory movements and phonetic content (9).

The baseline model employs a 5-layer GRU architecture trained with Connectionist Temporal Classification (CTC) loss (2) to predict phoneme sequences from neural activity. CTC is particularly well-suited for this task because it allows end-to-end training on unsegmented sequences where the alignment between neural activity and phoneme targets is unknown (2; 3). While this architecture achieves reasonable performance (∼20% PER), recent work has shown that improvements are possible through better training strategies, including learning rate optimization and context-aware phonetic representations such as diphones (10; 8).

Our work focuses on practical optimization: enabling faster experimentation cycles and better results on hardware accessible to most researchers. Our contributions are threefold: (1) We systematically analyze and optimize each component of the training pipeline, achieving cumulative speedups of 60% per epoch through mixed-precision training (6), efficient data loading, and memory-optimized padding strategies. (2) We demonstrate that architectural simplification which reduces both GRU

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

depth and input dimensionality through a learned projection bottleneck yields both faster training and better generalization. (3) We show that aggressive learning rate scheduling via OneCycleLR (7) enables rapid convergence, achieving 10.2× faster training to reach target accuracy thresholds.

2 Methods

2.1 Experimental Setup

All experiments were conducted on an NVIDIA RTX 4060 Laptop GPU (8GB VRAM) paired with an Intel i9-13900H CPU. This hardware constraint motivated our focus on memory-efficient techniques. The baseline configuration uses batch size 32, which represents the maximum that fits within 8GB VRAM for the original architecture. The dataset consists of 10,000 sentences from attempted speech recordings (10), with 8,800 sentences used for training and validation.

We evaluate using Phoneme Error Rate (PER). Note that the benchmark codebase refers to this metric as "CER" in variable names, but as the model outputs phonetic sequences, PER is the correct terminology and is used throughout this paper.

2.2 Training Pipeline Optimizations

Automatic Mixed Precision (AMP): We enabled PyTorch's native AMP (6) with FP16 training for forward and backward passes while maintaining FP32 precision for loss computation and optimizer states. Mixed-precision training reduces memory bandwidth requirements by storing activations and gradients in 16-bit floating-point format, enabling up to 2× memory savings. This allows larger batch sizes within the same memory budget. Additionally, modern GPU architectures like Ada Lovelace contain specialized tensor cores optimized for FP16 matrix multiplications, providing substantial computational speedup. The GradScaler component dynamically adjusts the loss scaling factor to prevent gradient underflow during backpropagation, maintaining numerical stability despite reduced precision (6).

DataLoader Optimization: We configured persistent workers (`num_workers=4`), enabled `pin_memory` for faster CPU-to-GPU transfers, and implemented prefetching to overlap data loading with computation. Persistent workers remain alive between training epochs, eliminating the overhead of spawning new processes for each epoch. Pinned memory allocates page-locked host memory, enabling asynchronous DMA transfers that reduce synchronization overhead. Together, these changes minimize CPU-GPU communication bottlenecks and reduce the time the GPU spends idle waiting for data.

Dynamic Padding with Alignment: Rather than padding sequences to arbitrary batch-maximum lengths, we implemented batch-wise dynamic padding where sequences are padded to the nearest multiple of 64 timesteps. This padding strategy serves dual purposes: (1) it reduces wasted computation on unnecessary padding tokens compared to fixed-length padding, and (2) aligning tensor dimensions to multiples of 64 enables better CUDA kernel performance, as modern GPUs optimize matrix operations for power-of-2 and moderate-multiple dimensions. This alignment allows PyTorch's autotuner to cache optimized kernel configurations across batches with similar shapes, reducing kernel launch overhead.

2.3 Architecture Modifications

We hypothesized that the baseline architecture was overparameterized, leading to slow training and potential overfitting to the limited training data. With only 8,800 training sentences and 56.7M parameters, the baseline model has approximately 6,400 parameters per training sample, which is a ratio that strongly favors memorization over generalization. Our modifications address this imbalance:

Reduced GRU Depth: We decreased from 5 to 4 GRU layers. The additional layer contributed marginally to representational capacity while significantly increasing computation time and memory usage.

Input Dimensionality Reduction via Learned Projection: The original model concatenates 32 time bins of 256 neural features, yielding 8,192-dimensional inputs to the GRU. We introduced a learned projection layer that compresses this to 1,024 dimensions before the GRU, incorporating Layer Normalization (1), GELU activation (4), and dropout.

With a hidden dimension of $H = 1024$, the original first GRU layer required $3\times8192\times1024 \approx 25.2$M parameters for the input-to-hidden gate weights alone. Our modification trades this for an 8.4M parameter projection layer (8192×1024) plus a smaller $3 \times 1024 \times 1024 \approx 3.1$M parameter GRU input gate, yielding a net reduction of approximately 13.7M parameters in the first layer. This bottleneck acts as implicit regularization while dramatically reducing computational cost.

Additional Efficiency Improvements: We made several targeted architectural refinements. The Gaussian smoothing kernel size was changed from 20 to 19; odd-numbered kernels enable symmetric padding (equal elements on both sides), which avoids asymmetric boundary artifacts and allows CUDA implementations to use optimized symmetric convolution routines. We consolidated the per-day adaptation layers into a single `nn.Parameter` tensor of shape (N_{days}, D, D) and perform the transformation via a batched tensor contraction $Y_{btk} = \sum_d X_{btd} W_{bdk}$, which is more memory-efficient than maintaining separate `nn.Linear` layers for each recording day. We also removed explicit GRU hidden state initialization, allowing PyTorch's optimized defaults to handle zero initialization more efficiently.

These combined changes reduced total parameters from 56.7M to 35.2M (38% reduction) while maintaining kernel length at 32 time steps for equivalent temporal context.

2.4 Optimizer and Learning Rate Schedule

AdamW Optimizer: We replaced Adam with AdamW (5), which decouples weight decay from the gradient-based update. In standard Adam, L2 regularization is implemented by adding the regularization term to the gradient, causing the weight decay rate to interact with the adaptive learning rate scaling. AdamW instead applies weight decay directly to the parameters:

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta\hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon) \tag{1}$$

where η is the learning rate, λ is the weight decay coefficient, and $\hat{m}_t$, $\hat{v}_t$ are the bias-corrected first and second moment estimates. This decoupling provides more consistent regularization behavior across parameters with different gradient magnitudes and has been shown to improve generalization (5).

OneCycleLR Schedule: We implemented the one-cycle learning rate policy (7), which enables the phenomenon of "super-convergence", allowing for faster training with better generalization using a single cycle of learning rate variation. The schedule consists of two phases: (1) **Warmup phase** (`pct_start` of total steps): Learning rate increases linearly from a small initial value to a maximum value. (2) **Annealing phase** (remaining steps): Learning rate decreases via cosine annealing to a minimum value (in our case 1/1000th of the maximum).

The high learning rate in the middle of training acts as a strong regularizer, helping the optimizer escape sharp local minima in favor of flatter regions of the loss landscape that generalize better (7). We selected `pct_start=0.10`, which dedicates 10% of training steps to warmup before reaching the maximum learning rate. This configuration balances rapid convergence with training stability.

For extended 10,000-step runs, we reduced the maximum learning rate from 1e-3 to 3e-4. With OneCycleLR's cosine annealing, the learning rate schedule is symmetric around the peak. At higher maximum learning rates with many training steps, the model spends a disproportionate fraction of training at learning rates too high for fine-grained parameter adjustments. Lowering the peak to 3e-4 compresses the high-LR phase, allowing the model to spend more time in the annealing regime where careful optimization occurs. This effectively transforms the schedule toward a Warmup-Stable-Decay (WSD) pattern, where the relatively flat middle portion of the cosine curve provides extended moderate-LR training.

Table 1: Incremental optimization results showing test CTC loss, PER, and training time per epoch. All improvements are cumulative.

Configuration	CTC Loss	PER	PER vs Prior	PER vs Base	Time (s)	Time vs Base
Baseline	1.3136	0.3694	—	—	522	—
+ AMP	1.3151	0.3683	-0.3%	-0.3%	375	-28.2%
+ DataLoader	1.3066	0.3669	-0.4%	-0.7%	362	-30.7%
+ Padding	1.3153	0.3711	$+1.1\%$	$+0.5\%$	335	-35.8%
+ Architecture	1.2332	0.3480	-6.2%	-5.8%	208	-60.2%
+ AdamW	1.0179	0.2989	-14.1%	-19.1%	207	-60.3%
+ OneCycleLR	0.9904	0.2879	-3.7%	-22.1%	207	-60.3%

3 Results

3.1 Incremental Optimization Impact

Table 1 presents the cumulative effect of each optimization on training time and validation PER. All measurements were taken at batch size 32 over 2,000 training steps to enable fair comparison.

The largest single improvement came from architecture modifications (-37.9% training time with -6.2% PER improvement), demonstrating that the original model was indeed overparameterized. The reduced input dimensionality (8,192 $\rightarrow$ 1,024) eliminates the computational bottleneck in the GRU input-to-hidden transformation, which dominates the per-timestep computation. AMP provided substantial time savings (-28.2%) with negligible accuracy impact, validating mixed-precision training for this application. The slight PER increase with padding ($+1.1\%$) is within measurement noise and is vastly outweighed by the 7.5% training speedup. The optimizer and scheduler changes primarily improved convergence speed and final accuracy rather than per-epoch computation time.

3.2 Learning Rate Schedule Analysis

We conducted ablations on the OneCycleLR `pct_start` parameter with different `pct_start` values ($\{0.0, 0.05, 0.1, 0.3\}$), which controls the fraction of training spent in the warmup phase.

Our experiments revealed that `pct_start` values of 0.05 and 0.10 (minimal warmup) achieved faster initial convergence and lower final PER compared to longer warmups (0.3). This suggests the model benefits from rapidly reaching high learning rates rather than gradual warmup, consistent with super-convergence behavior observed in computer vision tasks (7). The aggressive LR schedule works synergistically with the architectural bottleneck and dropout, which provide sufficient regularization to prevent divergence at high learning rates. We selected `pct_start=0.10` for our final experiments as it provided the best balance of stability and convergence speed.

3.3 Batch Size Scaling

Our optimizations reduced memory consumption sufficiently to enable batch sizes up to 128 on 8GB VRAM, a $4\times$ increase over the baseline maximum of 32. The memory savings come from three sources: (1) AMP's FP16 activations reduce activation memory by $\sim$50% (6), (2) the smaller model (35M vs. 57M parameters) reduces parameter and optimizer state memory by 38%, and (3) the reduced GRU input dimension (8,192 $\rightarrow$ 1,024) decreases the memory footprint of intermediate GRU computations.

To evaluate convergence speed across batch sizes, we conducted 10-minute training runs measuring time to reach various PER thresholds. Table 2 compares training wall-clock time across configurations.

At batch size 128, we achieve **10.2$\times$ speedup** to reach < 0.21 PER (6 minutes 51 seconds vs. 1 hour 9 minutes). The speedup increases at lower PER thresholds, suggesting our optimizations particularly benefit longer training runs where convergence efficiency matters most. Figure 1 visualizes this convergence behavior across batch sizes on a logarithmic time scale.

Larger batch sizes provide more stable gradient estimates, enabling the aggressive OneCycleLR schedule to work more effectively. The optimized BS 32 configuration did not reach the 0.21 PER

Table 2: Time to reach PER thresholds (speedup relative to Baseline BS 32).

	$<$ **0.30 PER**		$<$ **0.25 PER**		$<$ **0.21 PER**	
Configuration	Time	Speedup	Time	Speedup	Time	Speedup
Baseline (BS 32)	15m 39s	1.0×	28m 39s	1.0×	1h 09m	1.0×
Optimized (BS 32)	3m 17s	4.8×	6m 31s	4.4×	—*	—
Optimized (BS 64)	2m 05s	7.5×	4m 22s	6.6×	8m 13s	8.5×
Optimized (BS 128)	**2m 01s**	**7.8**×	**3m 57s**	**7.3**×	**6m 51s**	**10.2**×

*Did not converge to $<$ 0.21 PER within 10-minute test window.

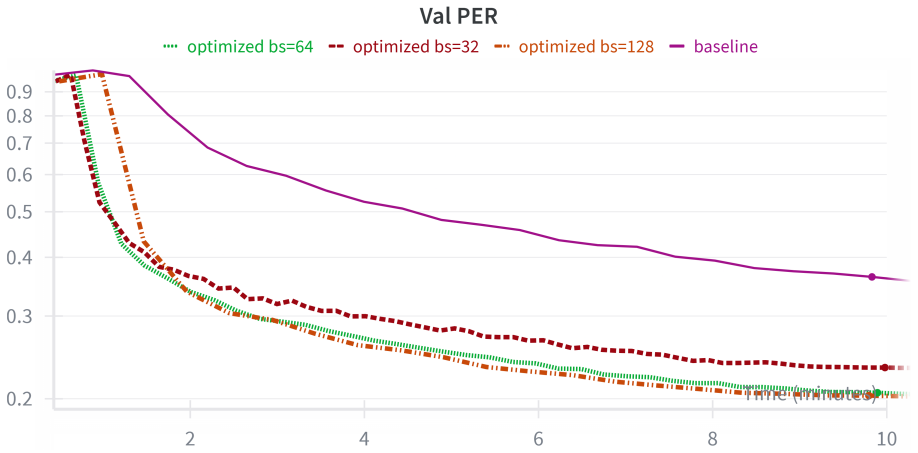

Figure 1: Convergence speed comparison across batch sizes. Time is shown in minutes on a logarithmic y-axis to clearly visualize the speedup differences. Larger batch sizes enabled by our optimizations achieve dramatic speedups, with BS 128 reaching target PER thresholds 10.2× faster than the baseline.

threshold within the 10-minute evaluation window, demonstrating that both architectural and batch size optimizations are necessary for maximal speedup.

3.4 Extended Training Results

We conducted a 10,000-step training run with the optimized configuration (batch size 128) using a reduced maximum learning rate of 3e-4 (vs. 1e-3 for shorter 2,000-step runs). Table 3 compares final performance metrics.

The optimized model achieves **17.18% PER**, a 16.4% relative error reduction compared to the baseline's 20.55% PER, despite using 38% fewer parameters (35.2M vs. 56.7M). Figure 2 shows the training and validation curves for both the baseline and optimized configurations throughout the extended training run.

4 Discussion

4.1 Why Smaller Models Perform Better

The counterintuitive result that a smaller model achieves better PER can be explained through the lens of the bias-variance tradeoff and implicit regularization. First, the reduced input dimensionality (8,192 $\rightarrow$ 1,024) forces the network to learn a compressed, more generalizable representation of the temporal neural features through the learned projection layer. This information bottleneck prevents the model from memorizing training-specific noise patterns in the high-dimensional neural recordings.

Table 3: Extended training comparison between baseline and optimized configurations.

	Baseline (Original)	Final Optimized (LR 3e-4)	Improvement
Lowest PER	0.2055	**0.1718**	**16.4% Relative Reduction**
Time to Best	74m 06s	**42m 02s**	**1.8× Faster**
Parameters	56,708,137	35,203,113	**38% Reduction**

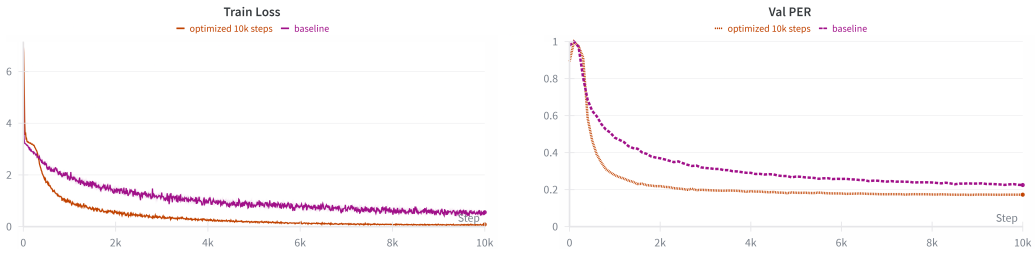

Figure 2: Extended training comparison between baseline and optimized models over 10,000 steps. **Left:** Training loss showing convergence behavior. **Right:** Validation PER demonstrating that the optimized model achieves both faster convergence and better final generalization (17.18% vs 20.55% PER) despite having 38% fewer parameters.

Second, with only 8,800 training sentences, the dataset is relatively small by modern deep learning standards. The baseline model's 56.7M parameters has sufficient capacity to memorize idiosyncratic patterns in this limited data, leading to poor generalization. Our 35.2M parameter model reduces the parameter to training sample ratio to approximately 4,000 parameters per sample, providing a more appropriate capacity for the available data. This is further supported by the reduced depth (4 vs. 5 layers), which simplifies the hypothesis space.

Third, fewer GRU layers may alleviate optimization difficulties during training with CTC loss. CTC requires the model to learn alignments between variable-length input sequences (neural activity) and variable-length output sequences (phonemes) without explicit supervision. Very deep RNNs can suffer from vanishing gradients, making it difficult for CTC's dynamic programming algorithm to effectively propagate error signals through both time and depth (2; 3). The shallower 4-layer architecture provides sufficient representational power while maintaining better gradient flow.

4.2 OneCycleLR and Super-Convergence

Our experiments with different `pct_start` values reveal that this task benefits from aggressive learning rate schedules with minimal warmup. The `pct_start=0.10` configuration, which spends only 10% of training in warmup before reaching maximum learning rate, achieved excellent results. This pattern is consistent with "super-convergence" phenomena (7), where high learning rates combined with strong regularization enable faster and better convergence.

The mechanism behind super-convergence involves the high learning rate acting as a regularizer that prevents the optimizer from settling into sharp, narrow minima of the loss landscape. Sharp minima correspond to overfitting because small perturbations to the inputs cause large changes in the loss. High learning rates cause the optimizer to "bounce out" of these sharp regions, favoring flatter minima where the loss is more stable under input perturbations. In our case, the architectural bottleneck (1,024-dim projection) and dropout (0.4) provide additional regularization that prevents divergence at high learning rates.

For extended training (10,000 steps), we found that reducing the maximum learning rate to 3e-4 prevented instability while maintaining the benefits of the one-cycle approach. The key insight is that OneCycleLR's cosine decay means the fraction of training spent at any given learning rate depends on the peak value. With a 1e-3 peak over 10,000 steps, the model spends thousands of steps at learning rates above 5e-4, which can cause oscillation around optima and prevent fine-grained parameter

tuning. Lowering the peak to 3e-4 ensures the model transitions more quickly to the fine-tuning regime (LR $<$ 1e-4) where precise adjustments can be made.

4.3 Practical Implications

Our results demonstrate that neural speech decoding research need not be limited to well-funded laboratories with access to high-performance computing clusters. An RTX 4060 Laptop GPU, commonly available in consumer laptops ($1,000–$1,500 price range), can achieve competitive results with proper optimization. This democratization of BCI research has important implications for reproducibility and broader participation in the field, particularly for researchers at institutions with limited computational budgets.

The 10.2$\times$ training speedup also enables more rapid experimentation. What previously required over an hour can now be accomplished in under 7 minutes, enabling researchers to test multiple hypotheses within a single work session. This acceleration is particularly valuable during the early exploratory phases of model development, where rapid iteration is essential for identifying promising approaches. For example, our `pct_start` ablations required only $\sim$30 minutes total, whereas on the baseline architecture they would have required several hours.

4.4 Limitations and Future Work

Our study has several limitations. First, we focused exclusively on the GRU architecture; Transformer-based approaches (8) may benefit differently from these optimizations, though recent competition results suggest transformers have not yet surpassed RNNs on this limited dataset size (10). Second, our experiments used the validation set for hyperparameter tuning; official test set performance requires submission to the competition website and was not evaluated in this study. Third, we did not explore word error rate (WER) evaluation, which requires additional language model integration and represents the ultimate metric for practical BCI applications.

Future directions include: (1) applying these optimization principles to Transformer architectures, potentially with adaptations for the irregular electrode spacing in neural recordings; (2) exploring additional augmentation strategies such as time-masking and frequency-masking commonly used in automatic speech recognition; (3) investigating context-dependent output representations such as diphones (8), which have shown promise by modeling phoneme transitions; (4) combining our efficient training pipeline with ensemble methods, which emerged as the most effective approach in the Brain-to-Text '24 competition (10); and (5) extending our methods to end-to-end brain-to-text architectures that integrate language modeling directly into the neural decoder rather than using a separate two-stage pipeline.

5 Conclusion

We presented a systematic optimization study of neural speech decoding on the Brain-to-Text Benchmark '24 dataset, achieving **10.2$\times$ faster training** to reach sub-0.21 PER and **16.4% relative error reduction** (from 0.2055 to 0.1718 PER) on consumer hardware. Our key findings are: (1) the baseline GRU model is substantially overparameterized for this 8,800-sentence dataset, and reducing model size through input projection and fewer layers improves both efficiency and accuracy; (2) aggressive learning rate scheduling with OneCycleLR enables super-convergence behavior when combined with appropriate architectural regularization; and (3) combined optimizations in mixed-precision training, efficient data loading, and architectural design have multiplicative benefits.

These results make neural speech decoding more accessible to researchers with limited computational resources while advancing performance on this important task. Our optimized model achieves 17.18% PER compared to the baseline's 20.55% PER. The first-place competition entry (DCoND) achieved 15.34% PER (8) by using context-aware diphone representations instead of single phonemes, combined with ensemble methods. Our work demonstrates that substantial improvements are achievable through training and architecture optimization alone, using a simpler single-model approach suitable for real-time applications.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 369–376, 2006.

[3] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. Speech recognition with deep recurrent neural networks. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6645–6649, 2013.

[4] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[6] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.

[7] Leslie N. Smith and Nicholay Topin. Super-convergence: Very fast training of neural networks using large learning rates. In *Proc. SPIE 11006, Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, pages 1100612, 2019.

[8] Jingyuan Li, Trung Le, Chaofei Fan, Mingfei Chen, and Eli Shlizerman. Brain-to-text decoding with context-aware neural representations and large language models. *Journal of Neural Engineering*, 22(5):056026, 2025.

[9] Francis R. Willett, Donald T. Avansino, Leigh R. Hochberg, Jaimie M. Henderson, and Krishna V. Shenoy. A high-performance speech neuroprosthesis. *Nature*, 620:1031–1036, 2023.

[10] Francis R. Willett, Erin M. Kunz, Joshua Rosenow, Krishna V. Shenoy, and Jaimie M. Henderson. Brain-to-Text Benchmark '24: Lessons learned. *arXiv preprint arXiv:2412.17227*, 2024.

Python Source Files

train_model.py

```
modelName = 'speechBaseline4'

args = {}
args['runName'] = 'optimized'
args['outputDir'] = '/mnt/d/c143a/data/speech_logs/' + modelName
args['datasetPath'] = '/mnt/d/c143a/data/ptDecoder_ctc'
args['seqLen'] = 150
args['maxTimeSeriesLen'] = 1200
args['batchSize'] = 128
args['lrStart'] = 1e-3
args['lrEnd'] = 1e-4
args['nUnits'] = 1024
args['nBatch'] = 2000 #3000
args['nLayers'] = 4
args['seed'] = 0
args['nClasses'] = 40
args['nInputFeatures'] = 256
args['dropout'] = 0.4
args['whiteNoiseSD'] = 0.8
args['constantOffsetSD'] = 0.2
args['gaussianSmoothWidth'] = 2.0
args['strideLen'] = 4
args['kernelLen'] = 32
args['bidirectional'] = False
args['l2_decay'] = 1e-5

from neural_decoder.neural_decoder_trainer import trainModel

trainModel(args)
```

model.py

```
import torch
from torch import nn

from .augmentations import GaussianSmoothing


class GRUDecoder(nn.Module):
    def __init__(
        self,
        neural_dim,
        n_classes,
        hidden_dim,
        layer_dim,
        nDays=24,
        dropout=0,
        device="cuda",
        strideLen=4,
        kernelLen=14,
        gaussianSmoothWidth=0,
        bidirectional=False,
    ):
        super(GRUDecoder, self).__init__()

        # Defining the number of layers and the nodes in each layer
        self.layer_dim = layer_dim
        self.hidden_dim = hidden_dim
        self.neural_dim = neural_dim
        self.n_classes = n_classes
        self.nDays = nDays
        self.device = device
        self.dropout = dropout
        self.strideLen = strideLen
        self.kernelLen = kernelLen
        self.gaussianSmoothWidth = gaussianSmoothWidth
        self.bidirectional = bidirectional
        self.inputLayerNonlinearity = torch.nn.Softsign()
        self.gaussianSmoother = GaussianSmoothing(
            neural_dim, 19, self.gaussianSmoothWidth, dim=1 # use 19 since odd number kernels
more efficient
        )
        self.dayWeights = torch.nn.Parameter(torch.randn(nDays, neural_dim, neural_dim))
        self.dayBias = torch.nn.Parameter(torch.zeros(nDays, 1, neural_dim))

        with torch.no_grad():
            for x in range(nDays):
                self.dayWeights.data[x, :, :] = torch.eye(neural_dim)

        # Instead of feeding (neural_dim * kernelLen) directly into the GRU (which creates
        # a massive weight matrix difficult to optimize), we project it down first.
        # This acts as a bottleneck and feature extractor.
        input_feat_dim = neural_dim * self.kernelLen

        self.input_projection = nn.Sequential(
            nn.Linear(input_feat_dim, hidden_dim),
            nn.LayerNorm(hidden_dim), # Stabilizes training significantly
            nn.GELU(),  # Modern activation function (better than Softsign/ReLU here)
            nn.Dropout(dropout)
        )
        # GRU layers
        self.gru_decoder = nn.GRU(
            hidden_dim,  # Input size is now hidden_dim (output of projection), not
input_feat_dim
            hidden_dim,
            layer_dim,
            batch_first=True,
            dropout=self.dropout,
            bidirectional=self.bidirectional,
```

```
        )

        for name, param in self.gru_decoder.named_parameters():
            if "weight_hh" in name:
                nn.init.orthogonal_(param)
            if "weight_ih" in name:
                nn.init.xavier_uniform_(param)
        nn.init.xavier_uniform_(self.input_projection[0].weight)
        nn.init.zeros_(self.input_projection[0].bias)
        output_dim = hidden_dim * 2 if self.bidirectional else hidden_dim
        self.fc_decoder_out = nn.Linear(output_dim, n_classes + 1)

    def forward(self, neuralInput, dayIdx):
        neuralInput = torch.permute(neuralInput, (0, 2, 1))
        neuralInput = self.gaussianSmoother(neuralInput)
        neuralInput = torch.permute(neuralInput, (0, 2, 1))

        # apply day layer
        dayWeights = torch.index_select(self.dayWeights, 0, dayIdx)
        dayBias = torch.index_select(self.dayBias, 0, dayIdx)
        transformedNeural = torch.einsum("btd,bdk->btk", neuralInput, dayWeights) + dayBias
        transformedNeural = self.inputLayerNonlinearity(transformedNeural)

        # stride/kernel
        stridedInputs = transformedNeural.unfold(1, self.kernelLen, self.strideLen)
        batch_size, num_windows, num_feats, kernel_len = stridedInputs.shape
        stridedInputs = stridedInputs.permute(0, 1, 3, 2).reshape(batch_size, num_windows,
num_feats * kernel_len)
        stridedInputs = self.input_projection(stridedInputs)

        # apply RNN layer
        hid, _ = self.gru_decoder(stridedInputs, None)

        # get seq
        seq_out = self.fc_decoder_out(hid)
        return seq_out
```

neural_decoder_trainer.py

```
import os
import pickle
import time

from edit_distance import SequenceMatcher
import hydra
import numpy as np
import torch
from torch.nn.utils.rnn import pad_sequence
from torch.utils.data import DataLoader
import wandb
from tqdm import tqdm
from torch.amp import autocast, GradScaler

from .model import GRUDecoder
from .dataset import SpeechDataset


def getDatasetLoaders(
    datasetName,
    batchSize,
):
    with open(datasetName, "rb") as handle:
        loadedData = pickle.load(handle)

    def _padding(batch):
        X, y, X_lens, y_lens, days = zip(*batch)

        # Instead of padding to exact batch max, pad to nearest multiple of 64
        # This keeps the "computation graph" more stable for kernel autotuning

        # 1. Calculate max lengths for this batch
        max_x_len = max([x.shape[0] for x in X])
        max_y_len = max([target.shape[0] for target in y])

        # 2. Round up to nearest multiple of 64
        pad_multiple = 64
        target_x_len = ((max_x_len + pad_multiple - 1) // pad_multiple) * pad_multiple
        target_y_len = ((max_y_len + pad_multiple - 1) // pad_multiple) * pad_multiple

        # 3. Manual Padding
        X_padded = []
        y_padded = []

        for x_item, y_item in zip(X, y):
            pad_size_x = target_x_len - x_item.shape[0]
            x_new = torch.nn.functional.pad(x_item, (0, 0, 0, pad_size_x), value=0)
            X_padded.append(x_new)
            pad_size_y = target_y_len - y_item.shape[0]
            y_new = torch.nn.functional.pad(y_item, (0, pad_size_y), value=0) # Y is likely 1D
[Time]
            y_padded.append(y_new)

        X_padded = torch.stack(X_padded)
        y_padded = torch.stack(y_padded)

        return (
            X_padded,
            y_padded,
            torch.stack(X_lens),
            torch.stack(y_lens),
            torch.stack(days),
        )

    train_ds = SpeechDataset(loadedData["train"], transform=None)
    test_ds = SpeechDataset(loadedData["test"])
    train_loader = DataLoader(
```

```
        train_ds,
        batch_size=batchSize,
        shuffle=True,
        num_workers=4,
        persistent_workers=True,
        pin_memory=True,
        collate_fn=_padding,
    )

    test_loader = DataLoader(
        test_ds,
        batch_size=batchSize,
        shuffle=False,
        num_workers=2,
        pin_memory=True,
        collate_fn=_padding,
    )
    return train_loader, test_loader, loadedData

def trainModel(args):
    print("Initializing Weights & Biases...")
    wandb.init(project="neural_seq_decoder_final", config=args, name=args.get("runName"))

    print("Setting up output directories and seeds...")
    os.makedirs(args["outputDir"], exist_ok=True)
    torch.manual_seed(args["seed"])
    np.random.seed(args["seed"])
    device = "cuda"

    with open(args["outputDir"] + "/args", "wb") as file:
        pickle.dump(args, file)

    print(f"Loading datasets from {args['datasetPath']}...")
    trainLoader, testLoader, loadedData = getDatasetLoaders(
        args["datasetPath"],
        args["batchSize"],
    )
    print(f"Data loaded. Train batches: {len(trainLoader)}, Test batches: {len(testLoader)}")

    print("Building model...")
    model = GRUDecoder(
        neural_dim=args["nInputFeatures"],
        n_classes=args["nClasses"],
        hidden_dim=args["nUnits"],
        layer_dim=args["nLayers"],
        nDays=len(loadedData["train"]),
        dropout=args["dropout"],
        device=device,
        strideLen=args["strideLen"],
        kernelLen=args["kernelLen"],
        gaussianSmoothWidth=args["gaussianSmoothWidth"],
        bidirectional=args["bidirectional"],
    ).to(device)

    wandb.watch(model, log="all", log_freq=100)
    print("Model built and moved to device.")

    loss_ctc = torch.nn.CTCLoss(blank=0, reduction="mean", zero_infinity=True)

    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=args["lrStart"],
        # weight_decay=args["l2_decay"], # does not seem to help
    )

    scheduler = torch.optim.lr_scheduler.OneCycleLR(
        optimizer,
        max_lr=args["lrStart"],
        total_steps=args["nBatch"],
        pct_start=0.1,
```

```
        anneal_strategy='cos',
    )

    scaler = GradScaler(device="cuda")

    print("Starting training loop...")
    testLoss = []
    testCER = []

    def run_validation(step_num, last_start_time):
        """Runs validation loop, calculates metrics, logs, and saves checkpoints."""
        with torch.no_grad():
            model.eval()
            allLoss = []
            total_edit_distance = 0
            total_seq_length = 0

            for X, y, X_len, y_len, testDayIdx in testLoader:
                X, y, X_len, y_len, testDayIdx = (
                    X.to(device),
                    y.to(device),
                    X_len.to(device),
                    y_len.to(device),
                    testDayIdx.to(device),
                )

                with autocast(device_type="cuda"):
                    pred = model.forward(X, testDayIdx)
                    loss = loss_ctc(
                        torch.permute(pred.log_softmax(2), [1, 0, 2]),
                        y,
                        ((X_len - model.kernelLen) / model.strideLen).to(torch.int32),
                        y_len,
                    )
                    loss = torch.sum(loss)

                allLoss.append(loss.cpu().detach().numpy())

                adjustedLens = ((X_len - model.kernelLen) / model.strideLen).to(torch.int32)
                for iterIdx in range(pred.shape[0]):
                    decodedSeq = torch.argmax(
                        pred[iterIdx, 0 : adjustedLens[iterIdx], :],
                        dim=-1,
                    )
                    decodedSeq = torch.unique_consecutive(decodedSeq, dim=-1)
                    decodedSeq = decodedSeq.cpu().detach().numpy()
                    decodedSeq = np.array([i for i in decodedSeq if i != 0])

                    trueSeq = np.array(
                        y[iterIdx][0 : y_len[iterIdx]].cpu().detach()
                    )

                    matcher = SequenceMatcher(
                        a=trueSeq.tolist(), b=decodedSeq.tolist()
                    )
                    total_edit_distance += matcher.distance()
                    total_seq_length += len(trueSeq)

            avgDayLoss = np.sum(allLoss) / len(testLoader)
            cer = total_edit_distance / total_seq_length

            endTime = time.time()
            tqdm.write(
                f"batch {step_num}, ctc loss: {avgDayLoss:>7f}, cer: {cer:>7f}, time elapsed: {
(endTime - last_start_time):>7.3f}"
            )
            wandb.log({"val_loss": avgDayLoss, "val_cer": cer}, step=step_num)


            if len(testCER) > 0 and cer < np.min(testCER):
```

```
                torch.save(model.state_dict(), args["outputDir"] + "/modelWeights")

            testLoss.append(avgDayLoss)
            testCER.append(cer)

            tStats = {}
            tStats["testLoss"] = np.array(testLoss)
            tStats["testCER"] = np.array(testCER)

            with open(args["outputDir"] + "/trainingStats", "wb") as file:
                pickle.dump(tStats, file)

            return time.time()

    startTime = time.time()
    train_iter = iter(trainLoader)
    pbar = tqdm(range(args["nBatch"]), desc="Training")

    for batch in pbar:
        model.train()

        try:
            X, y, X_len, y_len, dayIdx = next(train_iter)
        except StopIteration:
            train_iter = iter(trainLoader)
            X, y, X_len, y_len, dayIdx = next(train_iter)

        X, y, X_len, y_len, dayIdx = (
            X.to(device), y.to(device), X_len.to(device), y_len.to(device), dayIdx.to(device)
        )

        if args["whiteNoiseSD"] > 0:
            X += torch.randn(X.shape, device=device) * args["whiteNoiseSD"]

        if args["constantOffsetSD"] > 0:
            X += torch.randn([X.shape[0], 1, X.shape[2]], device=device) * args[
"constantOffsetSD"]

        with autocast(device_type="cuda"):
            pred = model.forward(X, dayIdx)
            loss = loss_ctc(
                torch.permute(pred.log_softmax(2), [1, 0, 2]),
                y,
                ((X_len - model.kernelLen) / model.strideLen).to(torch.int32),
                y_len,
            )
            loss = torch.sum(loss)

        optimizer.zero_grad()
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        scheduler.step()

        wandb.log({"train_loss": loss.item(), "lr": scheduler.get_last_lr()[0]}, step=batch)
        pbar.set_postfix({"loss": f"{loss.item():.4f}"})

        if batch % 100 == 0:
            startTime = run_validation(batch, startTime)

    run_validation(args["nBatch"], startTime)

    print("Training complete.")
    wandb.finish()


def loadModel(modelDir, nInputLayers=24, device="cuda"):
    modelWeightPath = modelDir + "/modelWeights"
    with open(modelDir + "/args", "rb") as handle:
        args = pickle.load(handle)
```

```
    model = GRUDecoder(
        neural_dim=args["nInputFeatures"],
        n_classes=args["nClasses"],
        hidden_dim=args["nUnits"],
        layer_dim=args["nLayers"],
        nDays=nInputLayers,
        dropout=args["dropout"],
        device=device,
        strideLen=args["strideLen"],
        kernelLen=args["kernelLen"],
        gaussianSmoothWidth=args["gaussianSmoothWidth"],
        bidirectional=args["bidirectional"],
    ).to(device)

    model.load_state_dict(torch.load(modelWeightPath, map_location=device))
    return model


@hydra.main(version_base="1.1", config_path="conf", config_name="config")
def main(cfg):
    cfg.outputDir = os.getcwd()
    trainModel(cfg)

if __name__ == "__main__":
    main()
```